New Horizons
Centro de Capacitación

# Práctica: Maestro-Detalle Dinámico con FormArray y Cálculo de Totales

*Paso a paso — Administrar partidas de gasto dinámicas con FormArray, cálculo reactivo en tiempo real y validación de reglas*

| | |
|---|---|
| **Curso** | Técnico de Desarrollo Full Stack con C# y Angular |
| **Sesión** | Sesión 10 — Formularios Reactivos y Validaciones Avanzadas |
| **Docente** | Edwin Barrientos |
| **Duración estimada** | 60–90 minutos |
| **Nivel** | Intermedio |
| **Entorno** | .NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express |

## 1. Objetivos

- Integrar FormArray dentro del FormGroup principal para modelar múltiples detalles de gasto.
- Implementar métodos dinámicos agregarFila() y eliminarFila(index) con validación mínima.
- Calcular subtotales de cada ítem (cantidad * precioUnitario) y el monto total acumulado con valueChanges.
- Validar que el formulario posea obligatoriamente al menos 1 partida con importes positivos.
- Alinear los contratos del formulario con CrearSolicitudDto y FluentValidation de C#.

## 2. Desarrollo paso a paso

### Paso 1 — Expandir SolicitudFormComponent con FormArray

Actualizamos la estructura del formulario en SolicitudFormComponent incorporando la colección detalles como FormArray:

**src/app/pages/solicitud-form/solicitud-form.component.ts**

```
// Añadir FormArray en el formulario
this.formulario = this.fb.group({
  motivo: ['', [Validators.required, Validators.minLength(10), Validators.maxLength(250)]],
  centroCostoId: [null, [Validators.required]],
  solicitanteId: [1],
  detalles: this.fb.array([], [Validators.required, Validators.minLength(1)])
});
```

## Paso 2 — Implementar Métodos de Fila Dinámica y Cálculo Automático

Añadimos la lógica de manipulación del FormArray y el cálculo del monto total en tiempo real:

`src/app/pages/solicitud-form/solicitud-form.component.ts`

```
get detalles(): FormArray {
  return this.formulario.get('detalles') as FormArray;
}

crearFilaDetalle(): FormGroup {
  return this.fb.group({
    concepto: ['', [Validators.required, Validators.minLength(3)]],
    cantidad: [1, [Validators.required, Validators.min(1)]],
    precioUnitario: [0, [Validators.required, Validators.min(0.1)]]
  });
}

agregarFila(): void {
  this.detalles.push(this.crearFilaDetalle());
}

eliminarFila(index: number): void {
  if (this.detalles.length > 1) {
    this.detalles.removeAt(index);
  }
}

calcularSubtotal(index: number): number {
  const fila = this.detalles.at(index);
  const cant = fila.get('cantidad')?.value || 0;
  const precio = fila.get('precioUnitario')?.value || 0;
  return cant * precio;
}

get montoTotalCalculado(): number {
  return this.detalles.controls.reduce((acc, control) => {
    const cant = control.get('cantidad')?.value || 0;
    const precio = control.get('precioUnitario')?.value || 0;
    return acc + (cant * precio);
  }, 0);
}
```

## Paso 3 — Renderizar la Tabla Dinámica de Gastos en el Template y CSS

Diseñamos la sección de partidas de gasto con iteración sobre el FormArray en solicitud-form.component.html y sus clases en solicitud-form.component.css:

**src/app/pages/solicitud-form/solicitud-form.component.html**

```
<div class="detalles-section" formArrayName="detalles">
  <div class="section-title-bar">
    <h3>Detalle de Gastos o Adquisiciones</h3>
    <button mat-stroked-button color="primary" type="button" (click)="agregarFila()">
      <mat-icon>add</mat-icon> Agregar Ítem
    </button>
  </div>

  <table class="items-table">
    <thead>
      <tr>
        <th class="col-concepto">Concepto</th>
        <th class="col-cantidad">Cantidad</th>
        <th class="col-precio">Precio Unitario</th>
        <th class="col-subtotal">Subtotal</th>
        <th class="col-accion"></th>
      </tr>
    </thead>
    <tbody>
      <tr *ngFor="let item of detalles.controls; let i = index" [formGroupName]="i">
        <td>
          <mat-form-field appearance="outline" class="dense-field">
            <input matInput formControlName="concepto" placeholder="Ej: Laptop Core i7">
          </mat-form-field>
        </td>
        <td>
          <mat-form-field appearance="outline" class="dense-field">
            <input matInput type="number" formControlName="cantidad" min="1">
          </mat-form-field>
        </td>
        <td>
          <mat-form-field appearance="outline" class="dense-field">
            <input matInput type="number" formControlName="precioUnitario" step="0.5" min="0">
          </mat-form-field>
        </td>
        <td class="subtotal-cell">
          {{ calcularSubtotal(i) | currency:'PEN':'symbol':'1.2-2' }}
        </td>
        <td>
          <button mat-icon-button color="warn" type="button" (click)="eliminarFila(i)"
[disabled]="detalles.length <= 1">
            <mat-icon>remove_circle</mat-icon>
          </button>
        </td>
      </tr>
    </tbody>
  </table>

  <div class="total-resumen">
    <span class="total-label">MONTO TOTAL SOLICITADO:</span>
    <span class="total-monto">{{ montoTotalCalculado | currency:'PEN':'symbol':'1.2-2' }}</span>
  </div>
</div>
```

**src/app/pages/solicitud-form/solicitud-form.component.css**

```
.form-container { max-width: 900px; margin: 0 auto; padding: 12px; }
.form-header { margin-bottom: 16px; }
.form-card { padding: 24px; border-radius: 12px; background: white; }
.full-width { width: 100%; margin-bottom: 12px; }
.field-row-split { display: flex; gap: 16px; margin-bottom: 16px; }
.half-width { flex: 1; }
.detalles-section { margin-top: 24px; border-top: 1px solid #e2e8f0; padding-top: 20px; }
.section-title-bar { display: flex; justify-content: space-between; align-items: center; margin-
bottom: 16px; }
.items-table { width: 100%; border-collapse: collapse; }
.items-table th { text-align: left; padding: 8px; color: #64748b; font-size: 0.85rem; }
.col-concepto { width: 45%; }
.col-cantidad { width: 15%; }
.col-precio { width: 20%; }
.col-subtotal { width: 15%; }
.col-accion { width: 5%; }
.dense-field { width: 100%; }
.subtotal-cell { font-weight: 600; padding: 8px; color: #1e293b; }
.total-resumen { display: flex; justify-content: flex-end; align-items: center; gap: 16px;
margin-top: 16px; padding: 16px; background: #f8fafc; border-radius: 8px; }
.total-label { font-weight: 600; color: #475569; }
.total-monto { font-size: 1.3rem; font-weight: 700; color: #024b98; }
.actions-bar { display: flex; justify-content: flex-end; gap: 12px; margin-top: 24px; }
```

### Paso 4 — Registrar la ruta /solicitudes/nueva en app.routes.ts

Actualizamos src/app/app.routes.ts para conectar el formulario reactivo dentro del Shell:

**src/app/app.routes.ts**

```
import { Routes } from '@angular/router';
import { ShellComponent } from './layout/shell/shell.component';
import { DashboardComponent } from './pages/dashboard/dashboard.component';
import { SolicitudesListComponent } from './pages/solicitudes-list/solicitudes-list.component';
import { SolicitudDetailComponent } from './pages/solicitud-detail/solicitud-detail.component';
import { SolicitudFormComponent } from './pages/solicitud-form/solicitud-form.component';

export const routes: Routes = [
  {
    path: '',
    component: ShellComponent,
    children: [
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
      { path: 'dashboard', component: DashboardComponent },
      { path: 'solicitudes', component: SolicitudesListComponent },
      { path: 'solicitudes/nueva', component: SolicitudFormComponent },
      { path: 'solicitudes/:id', component: SolicitudDetailComponent }
    ]
  },
  { path: '**', redirectTo: 'dashboard' }
];
```

## 3. Retos adicionales

1. Añade una validación cruzada personalizada para impedir guardar si el monto total excede los S/ 50,000.
2. Implementa la carga inicial de datos en el formulario cuando se accede en modo edición (/solicitudes/:id/editar).
3. Permite clonar un ítem existente del FormArray con un solo clic.
4. Agrega soporte para calcular automáticamente el IGV del 18% como desglose informativo.

## 4. Entregables

- FormArray integrado permitiendo añadir y remover filas de gasto dinámicamente.
- Cálculo en vivo de subtotales y total acumulado sin retardo.
- Bloqueo de envío cuando no existen detalles o cuando los importes son menores o iguales a cero.
- Contratos de datos listos para el consumo del endpoint POST /api/solicitudes de C#.